WEEK 05

Software Concurrency, Part 2

Fixing all three faults, one at a time. Tasks, a scheduler, interrupts, state machines, and finally letting the hardware do the waiting

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 3, printed pages 65–88.
Lab manual: Lab 04 (polling vs interrupt), Section 3.2.1 (modularisation and drivers).

1 What we are actually after this week

Last week we convicted the starter program on three counts: sluggish, loses events, spaghetti. This week we fix all three, in the order that makes each fix possible.

The sequence matters, so here is the plan and what each step buys:

1. **Tasks.** Split the program into three independent activities. Fixes *spaghetti*. Fixes responsiveness not at all, and in fact makes it marginally worse.
2. **A cooperative scheduler.** Six lines. Runs the tasks in a loop.
3. **Interrupts.** Move switch reading into a handler. Fixes *lost events* outright, and cuts the first half of the response delay from hundreds of milliseconds to hundreds of nanoseconds.
4. **The wrong fix**, shown deliberately, because the book shows it and because you will be tempted by it.
5. **Finite state machines.** Restructure tasks to return early. Fixes the second half of the response delay.
6. **Hardware timers.** Stop the CPU waiting at all. Fixes responsiveness *and* overhead together.
7. **Advanced topics**: task priorities, the waiting state, preemption, kernels, and real-time systems. Lighter treatment, but they are where this leads.

Items 3, 5, and 6 are the substance. Item 4 is the most educational failure in the chapter. This is a long one.

2 Deep dive 1: Tasks, which fix structure and nothing else

Start with the structural fault, because everything else is easier afterwards.

Task, and root function

> A **task** is a subroutine that performs a specific activity, or a closely related set of activities. Tasks simplify development by grouping related processing together and separating unrelated parts.
>
> A task's **root function** is its main function, which may call other functions as subroutines.

Our program has three activities hiding in it, so: three tasks.

Listing 1: Task 1: read the switches, publish what it found. Book Listing 3.4.

```
1 uint8_t   g_flash_LED = 0;
```

```
uint32_t g_w_delay   = W_DELAY_SLOW;
uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void Task_Read_Switches(void) {
    if (SWITCH_PRESSED(SW1_POS))  g_flash_LED = 1;    /* flash white  */
    else                          g_flash_LED = 0;    /* RGB sequence */

    if (SWITCH_PRESSED(SW2_POS)) {
        g_w_delay   = W_DELAY_FAST;
        g_RGB_delay = RGB_DELAY_FAST;
    } else {
        g_w_delay   = W_DELAY_SLOW;
        g_RGB_delay = RGB_DELAY_SLOW;
    }
}
```

Listing 2: Tasks 2 and 3. Each checks whether it has work before doing any. Book Listings 3.5, 3.6.

```
void Task_Flash(void) {
    if (g_flash_LED == 1) {                  /* only run in flash mode */
        Control_RGB_LEDs(1, 1, 1);
        Delay(g_w_delay);
        Control_RGB_LEDs(0, 0, 0);
        Delay(g_w_delay);
    }
}

void Task_RGB(void) {
    if (g_flash_LED == 0) {                  /* only run when NOT flashing */
        Control_RGB_LEDs(1, 0, 0);  Delay(g_RGB_delay);
        Control_RGB_LEDs(0, 1, 0);  Delay(g_RGB_delay);
        Control_RGB_LEDs(0, 0, 1);  Delay(g_RGB_delay);
    }
}
```

The tasks do not call each other. They communicate through three **global variables**, and the flow is one-directional: `Task_Read_Switches` writes all three, the other two tasks read from two each. Draw it as a diagram and you get producers and consumers with data in between, which is a structure you can reason about.

Notice the `g_` prefix on the globals. That is a convention, not a requirement, and it is worth adopting: it makes it obvious at every use site that this variable is shared, which is a warning you will want in week 8.

Beware the trap

> Global variables are how the book gets this working today, and the book says plainly that we will later learn why they are dangerous. Here is the preview, so you can start being uneasy now.
>
> `g_RGB_delay` is a `uint32_t`, and on this MCU a 32-bit load or store is a single instruction, so it is atomic. Fine. But make it a `uint64_t`, or a struct, or a pair of variables that must agree with each other, and a writer can be interrupted halfway through updating it, leaving a reader to observe a value that was never valid. Week 8 is entirely about this. For now, note that the safety of this design depends on a fact about instruction widths that nobody wrote down.

3 Deep dive 2: The scheduler

With tasks defined, the main loop becomes almost embarrassingly simple:

Listing 3: The whole scheduler. Book Listing 3.7.

```
void Flasher(void) {
    Init_GPIO_RGB();
    Init_GPIO_Switches();
    while (1) {
        Task_Read_Switches();
        Task_Flash();
        Task_RGB();
    }
}
```

That is it. Call each task in turn, forever.

Cooperative multitasking

> A scheduling approach where tasks share the CPU by *voluntarily yielding* it, which they do by returning. If one task takes a long time, it delays every other task by that much time. The scheduler has no power to interrupt a running task.
>
> Also called a **non-preemptive scheduler**: it cannot switch tasks until the running task completes.

3.1 The analysis: better structure, no better response

The program is now much better organised, and **its responsiveness is no better**. If anything it is a hair worse, because the scheduler adds three function calls per cycle that the old version did not have.

That is worth sitting with, because it is a genuine result rather than a disappointment. **Modularity and responsiveness are independent axes.** We spent effort and bought only one of them. The switch press is still only noticed when `Task_Read_Switches` gets to run, which is still after `Task_RGB` finishes its 1200 ms of delays.

But the fix has bought us something invisible: a place to stand. Now that switch reading is a separate thing, we can move it somewhere else entirely.

3.2 Splitting the delay into T_1 and T_2

Before we do, let's decompose the response time properly, because the two halves have different cures. The delay from moving the switch to seeing the LED change has two parts:

	What it is	Fixed by
T_1	Switch changes → `g_flash_LED` gets updated	Interrupts (Section 3)
T_2	`g_flash_LED` updated → LED behaviour changes	Shorter tasks (Sections 5, 6)

T_1 is large when you press during a long task, because `Task_Read_Switches` cannot run until that task finishes. T_2 is small in our task ordering because `Task_Flash` happens to run immediately after `Task_Read_Switches`, and it is short.

And notice that T_2 depends on the *order* of the tasks in the scheduler. Reorder them so `Task_RGB` sits between the reader and `Task_Flash`, and T_2 balloons. Task order is a design decision with a measurable consequence, which is the seed of the prioritisation idea in Section 7.

In the lab

Worked example, from the book's Exercise 1. Suppose the tasks have these execution times:

Task	In flash mode	In RGB mode
`Task_Read_Switches`	1 ms	1 ms
`Task_Flash`	100 ms	1 ms
`Task_RGB`	1 ms	1000 ms

To find the *maximum* delay between pressing the switch and seeing white, reason about the worst possible moment to press. The system is in RGB mode, so a full cycle is $1 + 1 + 1000 = 1002$ ms. The worst instant is immediately *after* `Task_Read_Switches` samples the pin, because you then have to wait out everything before it gets to sample again.
So: the rest of the reader (≈ 0), then `Task_Flash` in RGB mode (1 ms, does nothing), then `Task_RGB` (1000 ms), then the reader runs again (1 ms) and sets the flag, and only then does `Task_Flash` light the LED white. Total $\approx$ **1002 ms**.

Now do part (b) yourself: the maximum delay between *releasing* the switch and seeing the RGB sequence resume. The method is identical, but you are now starting in flash mode, so a full cycle has different numbers, and the task that must complete before the reader runs is a different one. Work out which instant is worst and add up what stands between it and `Task_RGB` producing output. If your answer is not noticeably smaller than 1002 ms, check which mode you assumed.

4 Deep dive 3: Interrupts, which fix T_1

Recall the distinction from last week: polling means your code asks whether work is needed, and event triggering means the hardware tells you. The hardware mechanism that does the telling is the interrupt system.

IRQ and ISR

An **interrupt request (IRQ)** is a hardware signal indicating that an interrupt is requested. A peripheral raises one when an event occurs.

An **interrupt service routine (ISR)**, also called a handler, is the software routine that runs in response.

The sequence: the CPU finishes the current instruction, saves the program's state, jumps to the ISR, runs it, restores the saved state, and resumes at the next instruction. The interrupted code never knows it happened.

So instead of `Task_Read_Switches` being called by the scheduler, the switch hardware calls it, at the instant the switch moves.

Listing 4: Switch reading moved into an ISR. Book Listing 3.8, from `Flasher3/main.c`.

```
volatile uint8_t  g_flash_LED = 0;
volatile uint32_t g_w_delay   = W_DELAY_SLOW;
volatile uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void EXTI4_15_IRQHandler(void) {
    if (EXTI->PR & MASK(SW1_POS)) {
```

```
        EXTI->PR = MASK(SW1_POS);           /* ack: write 1 to CLEAR the bit
            */
        if (SWITCH_PRESSED(SW1_POS))  g_flash_LED = 1;
        else                          g_flash_LED = 0;
    }
    if (EXTI->PR & MASK(SW2_POS)) {
        EXTI->PR = MASK(SW2_POS);           /* ack */
        if (SWITCH_PRESSED(SW2_POS)) {
            g_w_delay = W_DELAY_FAST;   g_RGB_delay = RGB_DELAY_FAST;
        } else {
            g_w_delay = W_DELAY_SLOW;   g_RGB_delay = RGB_DELAY_SLOW;
        }
    }
    NVIC_ClearPendingIRQ(EXTI4_15_IRQn);
}
```

And the scheduler loses a line:

```
void Flasher(void) {
    Init_GPIO_RGB();
    Init_GPIO_Switches_Interrupts();
    while (1) {
        Task_Flash();
        Task_RGB();
    }
}
```

Three details in that handler are worth naming carefully, because each is a classic bug.

One: the globals became `volatile`. Mandatory now. The compiler can see that `Task_RGB` reads `g_flash_LED` in a loop and that nothing in `Task_RGB` writes it, so without `volatile` it is entitled to read it once and cache it in a CPU register forever. Your ISR would update memory and the task would never notice. Recall the `__IO` qualifier from Week 03: same idea, same reason.

Two: acknowledging by writing a one. `EXTI->PR` is a pending register, and you clear a pending bit by *writing a one to it*, not a zero. This feels backwards and it is deliberate: it means you can clear exactly the bit you want with a single write and no read-modify-write, exactly like `BSRR` in Week 03. Same design philosophy, and the same benefit, atomicity.

Three: if you forget to acknowledge, the ISR runs forever. The pending flag stays set, so the moment the handler returns, the interrupt fires again immediately, and your program does nothing but service the same interrupt until you unplug it. This is the single most common interrupt bug.

4.1 The analysis: T_1 collapses

Two wins.

First, the switch-reading code now runs *only when a switch actually moves*, which frees up whatever fraction of the CPU it was consuming and, more importantly, means a brief press can no longer be missed. The hardware latches the edge. Fault 2 from last week is now gone entirely, not mitigated.

Second, T_1 becomes tiny. How tiny?

Book board vs your board

> **Interrupt latency**, from the request arriving to the first instruction of the handler executing:
>
> Book's Cortex-M0 at 48 MHz: **16 cycles**, which is $16/48\,\text{MHz} = \mathbf{333.3\ ns}$.
> Your Cortex-M4 at 72 MHz: **12 cycles**, which is $12/72\,\text{MHz} \approx \mathbf{167\ ns}$.
>
> Compare either of those to the 1002 ms we computed for the polled version. That is

a factor of roughly three million. The M4 is faster both because it needs fewer cycles (ARMv7-M has a more capable exception entry path, plus tail-chaining) and because it runs at a higher clock. Recall the NVIC priority-bit difference from Week 01: that same richer interrupt controller is why.

The standard division of labour

For most embedded systems a combination of polling and ISRs is enough. Urgent processing goes in the ISR. Less urgent work happens on a polling basis in the background. Often the ISR does the minimum urgent part and **saves partial results for later, slower processing** elsewhere.

That last pattern is called *deferred processing*, and it is the single most useful structural idea in interrupt-driven design.

But T_2 is untouched. The scheduler is still non-preemptive, so once `Task_RGB` has started its thousand milliseconds of delay, nothing short of a reset is going to hurry it along.

5 Deep dive 4: The wrong fix, shown on purpose

Here is where the book does something unusually honest: it writes the bad solution out in full so you can see how bad it gets.

The obvious idea is to test `g_flash_LED` more often inside `Task_RGB`, so it can bail out early:

Listing 5: "A very bad idea." Book Listing 3.10, abridged, and yes it really looks like this.

```
if (g_flash_LED == 0) { Delay(g_RGB_delay); }
if (g_flash_LED == 0) { Control_RGB_LEDs(0, 0, 1); }
if (g_flash_LED == 0) { Delay(g_RGB_delay); }
```

Messy, inefficient, and, critically, **it does not even work**. If the switch is pressed while `Delay` is running, the task still cannot finish until `Delay` returns. The guard tests sit *between* the delays, and the delays are where all the time goes.

So the natural next move is to push the test down into `Delay` itself. But both tasks call `Delay`, and they want to bail out on opposite conditions. So you need either two versions of `Delay`, or a parameter telling it who called it:

Listing 6: The book's own verdict on this: "a wonderful example of spaghetti code and how not to do things."

```
void Delay(unsigned int time_del, int called_by_Task_Flash) {
    /* ... a general-purpose delay function that now knows about
           the specific tasks that call it. Everything is coupled
           to everything. */
}
```

Beware the trap

Look at what just happened, because the pattern recurs everywhere. We tried to buy responsiveness by adding guard clauses, and we paid for it in modularity, and then *we did not even get the responsiveness*. A general-purpose utility function now takes a parameter naming one of its callers. Any new task means editing `Delay`.

The lesson is not "avoid guard clauses". It is that when a fix requires a low-level utility to know about high-level callers, the design is wrong and the correct move is to restructure, not to patch. That restructuring is the next section.

6 Deep dive 5: Finite state machines, which fix T_2

We want a task that **returns before it has finished all its work**, so the scheduler gets more frequent chances to run something else. The clean structure for that is a state machine.

Finite state machine (FSM)

> A state machine with all states and transitions defined. It executes the code of *one state* each time it is called, then returns. Progress is tracked in a state variable that survives between calls.
>
> The recipe: break the code into states **after each long-running operation**. If there are loops or conditionals, each state must start and end at the same level of nesting.

For `Task_RGB` the three delays give three states:

State	Action	Next state on time-out
`ST_RED`	Light red LED	`ST_GREEN`
`ST_GREEN`	Light green LED	`ST_BLUE`
`ST_BLUE`	Light blue LED	`ST_RED`

That table is a **state transition table**, and the equivalent picture, circles for states with labelled arrows for transitions, is a **state transition diagram**. An arrow labelled "reset" marks the initial state. An unlabelled arrow fires automatically when the state completes. Learn to draw these, because they are how you communicate a design to another engineer, and they turn up in exams as "draw the state diagram for..." questions.

Listing 7: FSM version. Book Listing 3.12. The `static` is the whole trick.

```
void Task_RGB_FSM(void) {
    enum State { ST_RED, ST_GREEN, ST_BLUE };
    static enum State next_state;

    if (g_flash_LED == 0) {
        switch (next_state) {
        case ST_RED:
            Control_RGB_LEDs(1, 0, 0);
            Delay(g_RGB_delay);
            next_state = ST_GREEN;
            break;
        case ST_GREEN:
            Control_RGB_LEDs(0, 1, 0);
            Delay(g_RGB_delay);
            next_state = ST_BLUE;
            break;
        case ST_BLUE:
            Control_RGB_LEDs(0, 0, 1);
            Delay(g_RGB_delay);
            next_state = ST_RED;
            break;
        default:
            next_state = ST_RED;
            break;
        }
    }
}
```

Three C features are doing real work here and each is worth understanding rather than copying.

`static` on `next_state`. Without it, the variable is an ordinary local, created fresh on the stack each call, and the machine would reset to state zero every single time. `static` gives a local variable *static storage duration*: it lives for the whole program, keeping its value between calls, while its *visibility* stays confined to this function. That combination is exactly what a state machine needs.

`enum` for the states. It defines a type whose values are limited to the listed names, so `ST_RED` is a readable name for 0. Costs nothing at run time and makes the `switch` self-documenting.

The `default` case. Not decoration. If `next_state` ever holds a value outside the enum, through memory corruption or a missing initialiser, `default` recovers to a known state instead of falling through and doing nothing forever. This is the reliability attribute from Week 01 appearing as three lines of C.

`Task_Flash` gets the same treatment with two states, `ST_WHITE` and `ST_BLACK`. Note that the two `next_state` variables in the two functions are completely separate. They are **local variables**, visible only inside the function that declares them, so nothing outside can touch either one. That is the modularity we were trying to protect in Section 4, preserved this time.

And the scheduler does not change at all beyond calling the new function names. That is the sign of a good refactor.

6.1 The analysis

Better. Much better. Press the switch while green is lit and the flashing now starts after the *current* stage of the sequence rather than after the last one. T_2 has dropped from "the rest of the whole cycle" to "the rest of one colour".

And it is tunable: if one colour's delay is still too long, split it into two states and halve the worst case again. You are trading response time against the number of states, which is a knob you now control.

But we are still burning the entire CPU inside `Delay`. Which brings us to the good part.

7 Deep dive 6: Let the hardware wait

Recall from Week 01 that peripherals are specialised hardware that offloads work from the CPU. And recall that a busy-wait delay is 100% CPU overhead by definition, since waiting is the canonical example of doing no useful work.

So: stop making the CPU wait. Make a timer wait.

Timer, at the level we need it this week

> At heart a **counter** circuit that counts pulses. Feed it pulses of known frequency and it measures time. It can generate an event after a set delay, measure a pulse width, generate pulse outputs, or count external events.
>
> The processor could do all this in software, with much greater complexity and much less accuracy, and it might have to spend *all* its time on that one activity to get adequate precision.

The book's arrangement, which is a preview of week 12: a 48 MHz clock feeds a **prescaler** set to divide by 4800, giving 48 MHz / 4800 = **1 kHz**, so one pulse per millisecond. Those pulses feed a **down-counter**. You load a count, start it, and each pulse decrements it. At zero the timer sets a flag in a control register. Software reads that flag with `TIM_Expired()` and stops the timer with `Stop_TIM()`.

So a "delay of 150" becomes "load 150, start, and go do something else for a while".

7.1 Wait states

Restructuring for this needs one new idea: every delay becomes a **wait state** that the FSM sits in without blocking. Three colours plus three waits gives six states.

Listing 8: FSM with hardware timer. Book Listing 3.14, the Flash task version.

```
void Task_Flash_FSM_Timer(void) {
    enum State { ST_WHITE, ST_WHITE_WAIT, ST_BLACK, ST_BLACK_WAIT };
    static enum State next_state = ST_WHITE;

    if (g_flash_LED == 1) {
        switch (next_state) {
        case ST_WHITE:
            Control_RGB_LEDs(1, 1, 1);
            Start_TIM(g_w_delay);              /* fire and forget */
            next_state = ST_WHITE_WAIT;
            break;
        case ST_WHITE_WAIT:
            if (TIM_Expired()) {               /* not expired? just return */
                Stop_TIM();
                next_state = ST_BLACK;
            }
            break;
        case ST_BLACK:
            Control_RGB_LEDs(0, 0, 0);
            Start_TIM(g_w_delay);
            next_state = ST_BLACK_WAIT;
            break;
        case ST_BLACK_WAIT:
            if (TIM_Expired()) {
                Stop_TIM();
                next_state = ST_WHITE;
            }
            break;
        default:
            next_state = ST_WHITE;
            break;
        }
    } else {
        next_state = ST_WHITE;                 /* reset on mode change */
    }
}
```

Look at `ST_WHITE_WAIT` closely, because that case is the entire point of the exercise. It checks a flag. If the flag is clear it does nothing and **returns immediately**. No delay, no loop, nothing. The task is "waiting" and yet costs microseconds per call.

7.2 The analysis

Now every call to every FSM completes in essentially no time, including the calls that land in a wait state, because a hardware timer is tracking the delay rather than a software loop on the CPU. Response time is now bounded by roughly one trip through the scheduler, which is microseconds.

We have fixed all three original faults:

Original fault	Fixed by	Result
Spaghetti structure	Tasks + scheduler	Three independent activities
Lost events	Interrupt on switch edges	Hardware latches every edge
Sluggish, T_1	Interrupt	1002 ms $\rightarrow \approx$ 167 ns
Sluggish, T_2	FSM + hardware timer	Full cycle $\rightarrow$ one scheduler pass

In the lab

This is exactly the arc of your Lab 04, and it is worth seeing that the lab is a compressed version of this chapter. **Task 1** blinks an LED using a timer with **polling**: you start the timer and check its update flag in a loop, which is the `ST_WHITE_WAIT` pattern above but without the FSM, so the loop is still trapped. **Task 2** does the same job with a **timer interrupt**, which is the endgame: the CPU is not involved in timing *or* in noticing that the timing finished.

When you write that lab report, the interesting question is not whether the LED blinked. It is what your `while(1)` loop was free to do in each version. Write that comparison down, because it is what the viva will probe.

It also matters for your self-balancing robot. A balancing loop must sample the gyro, filter, compute PID, and update motors at a fixed rate, perhaps every 5 ms. If any part of that uses `HAL_Delay`, you have lost, because the delay time is time the controller is not controlling and the robot is falling over. Every wait in that project must be a timer, and preferably a timer interrupt.

8 Deep dive 7: What is still wrong, and where this leads

Three problems remain, and the book is upfront that fixing them properly is beyond an introductory text. But you should know their names, because they are the vocabulary of RTOS work and of the follow-on Embedded Systems elective.

8.1 Problem 1: the CPU still polls tasks that have no work

Look at the scheduler. It calls `Task_Flash_FSM_Timer` millions of times per second, and almost every call immediately returns because `g_flash_LED` is 0 or the timer has not expired. The program only does useful work when a switch changes or an LED changes. Everything else is overhead.

The fix is a third task state:

Task states, and blocking

Two states so far: **running** (at most one per core) and **ready to run** (any number).

Add **waiting**, also called **blocking**: the task is waiting for a specific event, a delay to expire, a message to arrive, a resource to free up. The scheduler **does not even attempt** to run waiting tasks. When the event occurs, the kernel moves the waiting tasks to ready.

Compare that with our FSM wait state. We got waiting by restructuring the code by hand into explicit wait states. With kernel support you do not restructure at all: you call a wait function, and from the task's point of view it simply does not return until the event happens. Same behaviour, far less code surgery, at the cost of needing a kernel underneath.

Scheduler, kernel, RTOS

A **scheduler** decides which task runs. Add task-oriented features (synchronisation, signalling, communication, time delays) and you have a **kernel**. Its fundamental job is to run *the highest priority task that is ready*. A **real-time kernel (RTK)** or **RTOS** is one built to execute with consistent, predictable timing rather than merely good average performance.

8.2 Problem 2: the tasks always run in the same order

Task order sets T_2, and here is the sting: **there is no order that is best for both directions**.

To minimise the response to *pressing* the switch, you want `Task_Flash` to run before `Task_RGB`. To minimise the response to *releasing* it, you want `Task_Flash` to run after `Task_RGB`. One fixed order cannot satisfy both, and no amount of cleverness in the tasks changes that.

Priorities are the general form of this decision. They can be **fixed** (task A is always lowest) or **dynamic** (recalculated at run time). Embedded kernels typically offer fixed priorities, some allow changing them while running.

8.3 Problem 3: an urgent event just after a long task starts

Even with priorities, a non-preemptive scheduler must wait for the running task to yield. The FSM approach solves this by making tasks short, but that is code surgery on every task.

Preemptive multitasking

The scheduler can **halt a running task partway through** to run a more urgent one, then resume the first. So an urgent task is not delayed by a lower-priority task that happens to be running.

Typical trigger: an ISR runs, defers work to a high-priority task, and that task preempts whatever was running.

Concretely: with cooperative scheduling, pressing the switch during the green stage means the ISR sets the flag and then `Task_RGB` carries on regardless, finishing green and blue. With preemption, `Task_Flash` runs immediately after the ISR, and `Task_RGB` resumes only once `Task_Flash` is waiting again.

Beware the trap

Preemption is the most responsive option and it introduces the hardest class of bug in embedded work. Once a task can be halted at an arbitrary instruction, it can be halted *halfway through updating a shared variable*, and another task can then read a value that was never valid. Every global variable in your program becomes a hazard.

This is not hypothetical and it is not rare, and it is why Week 08 is devoted to critical sections and safe data sharing. Do not reach for preemption before you have read that chapter.

8.4 Real-time systems

Real-time system

A system in which tasks have **deadlines**. If the software and hardware are not responsive enough, a task misses its deadline and **that is a system failure**, not a performance complaint.

Real-time scheduling analysis gives mathematical methods to compute the worst-case

> response time of each task, which you then compare against the deadlines to determine whether the system is **schedulable**, meaning it will always meet them.

If it is not schedulable, you have a bounded set of moves, and it is a good list to have memorised because it is a natural exam question:

- Change the hardware, use a faster processor, if the budget allows.
- Speed up the application code, or reduce how much processing is needed.
- Rebalance work between ISRs and deferred activities.
- Add or change task priorities.
- Add preemption.

Recall the constraints list from Week 01 and notice that the first option spends money, the second spends engineering time, and the last three spend complexity. Same trade-offs, one chapter later, now with names.

And that is a wrap on concurrency. Pat yourself on the back, this was the longest chapter yet.

9 Tying it back

We started last week with a program that was sluggish, dropped inputs, and was a tangle. This week we fixed all three, and the useful thing is that each fix addressed exactly one fault.

Tasks and a scheduler fixed the structure and bought no responsiveness whatsoever, which taught us that modularity and responsiveness are separate axes. **Interrupts** eliminated lost events outright, because hardware latches an edge whether or not your code was looking, and collapsed T_1 from a full scheduler cycle to about 167 nanoseconds on your Cortex-M4. Then we watched the **wrong fix** for T_2 destroy the modularity we had just built, and learned that a fix requiring a utility function to know its callers' names is a sign to restructure rather than patch. **Finite state machines** restructured correctly, using `static` to remember progress across calls so a task could return early. And **hardware timers** removed the waiting from the CPU altogether, which is the only move in the chapter that improved responsiveness and overhead at the same time.

What remains is a scheduler that still polls idle tasks, a fixed task order that cannot be optimal in both directions, and no way to interrupt a long task. Those get solved by a kernel with a blocking state, task priorities, and preemption, which is where an RTOS comes from and which is the elective after this course.

Next week we go down a level. We have been using interrupts as a black box: something fires and the handler runs. Week 6 opens the CPU itself and looks at registers, memory layout, and instruction encoding, so that in week 7 we can explain what "the processor saves the program's current information" actually means, register by register.

Cheat sheet: Week 05

The fix table, memorise this shape

Tasks + scheduler → modularity only. Response time unchanged, slightly worse.
Interrupts → kills lost events, collapses T_1.
FSM → shortens T_2 by letting a task return before finishing.
Hardware timer → responsiveness *and* overhead together. Best move available.
T_1 = switch change → variable updated. T_2 = variable updated → LED changes.

Definitions

Task: subroutine performing one activity. Its **root function** is its entry point.
Cooperative multitasking: tasks yield the CPU voluntarily by returning. Same thing as a **non-preemptive scheduler**.
IRQ: hardware signal requesting an interrupt. **ISR**: the routine that runs in response.
FSM: state machine with all states and transitions defined. Runs one state per call.
Waiting / blocking: task state where it awaits an event; the scheduler does not try to run it.
Preemptive multitasking: scheduler can halt a running task to run a more urgent one.
Real-time system: tasks have deadlines, and a miss is a failure. **Schedulable** = always meets them.
Kernel = scheduler + synchronisation, signalling, communication, delays.

Interrupt latency

Cortex-M0 at 48 MHz: 16 cycles = **333.3 ns**. Cortex-M4 at 72 MHz: 12 cycles ≈ **167 ns**.
Standard division of labour: urgent work in the ISR, less urgent work polled in the background, and the ISR **defers** slow processing by saving partial results.

Three ISR rules that cause real bugs

1. Variables shared with an ISR **must be** `volatile`, or the compiler caches them in a register and the update is never seen.
2. Clear a pending flag by **writing a 1** to it (`EXTI->PR = MASK(n);`). Same atomic-write philosophy as `BSRR`.
3. **Forget to acknowledge and the ISR re-fires forever.** Most common interrupt bug there is.

FSM construction recipe

Break the code into states **after each long-running operation**. Each state must start and end at the same nesting level.
`static enum State next_state;` → `static` gives it lifetime across calls while keeping it local in scope. Drop `static` and the machine resets every call.
Always include a `default:` case that recovers to a known state.
For hardware-timer delays, add one **wait state** per delay: start the timer in one state, poll `TIM_Expired()` in the next and return immediately if it is clear.
Document it as a **state transition table** or **diagram**. Both turn up in exams.

Why one task order cannot win

To respond fastest to **press**, `Task_Flash` must run *before* `Task_RGB`.
To respond fastest to **release**, it must run *after*. No single fixed order does both. That is the argument for priorities, and ultimately for preemption.

If the system is not schedulable, five moves

Faster processor (costs money) · faster code or less processing (costs engineering) · rebalance ISR vs deferred work · add or change priorities · add preemption (all three cost complexity).

Likely exam prompts from this chapter

· Given per-task execution times, compute the maximum delay from an event to a visible response. *Very likely, this is the book's Exercise 1.*
· Convert a task with delays into an FSM, and draw its state transition diagram.
· Why must `next_state` be `static`? Why must shared variables be `volatile`?
· Distinguish cooperative from preemptive scheduling, and name one risk preemption introduces.
· Explain how a hardware timer improves both responsiveness and CPU overhead.
· Define a real-time system and state what "schedulable" means.